Higher Colleges of Technology

كليات التقنية العليا

CDS 2413- PROGRAMMING FOR DATA SCIENCE

Week 11-12

Week 11	CLO 3: Create scripts to solve business problems using data analysis techniques such as classification, clustering, optimization, and recommendation

Document Revision CONTROL (DRC)

Version	Author	Effective Date	Change Description	DRC No
1.0	Mohammed Abdul Rahim	2/05/2023	Define the first version	001

X.Y
X: Major change
Y: minor change

Lecture Outline

- Clustering
- Types of Clustering
- Theory behind Clustering
- Python Programming for Clustering

LECTURE OBJECTIVES

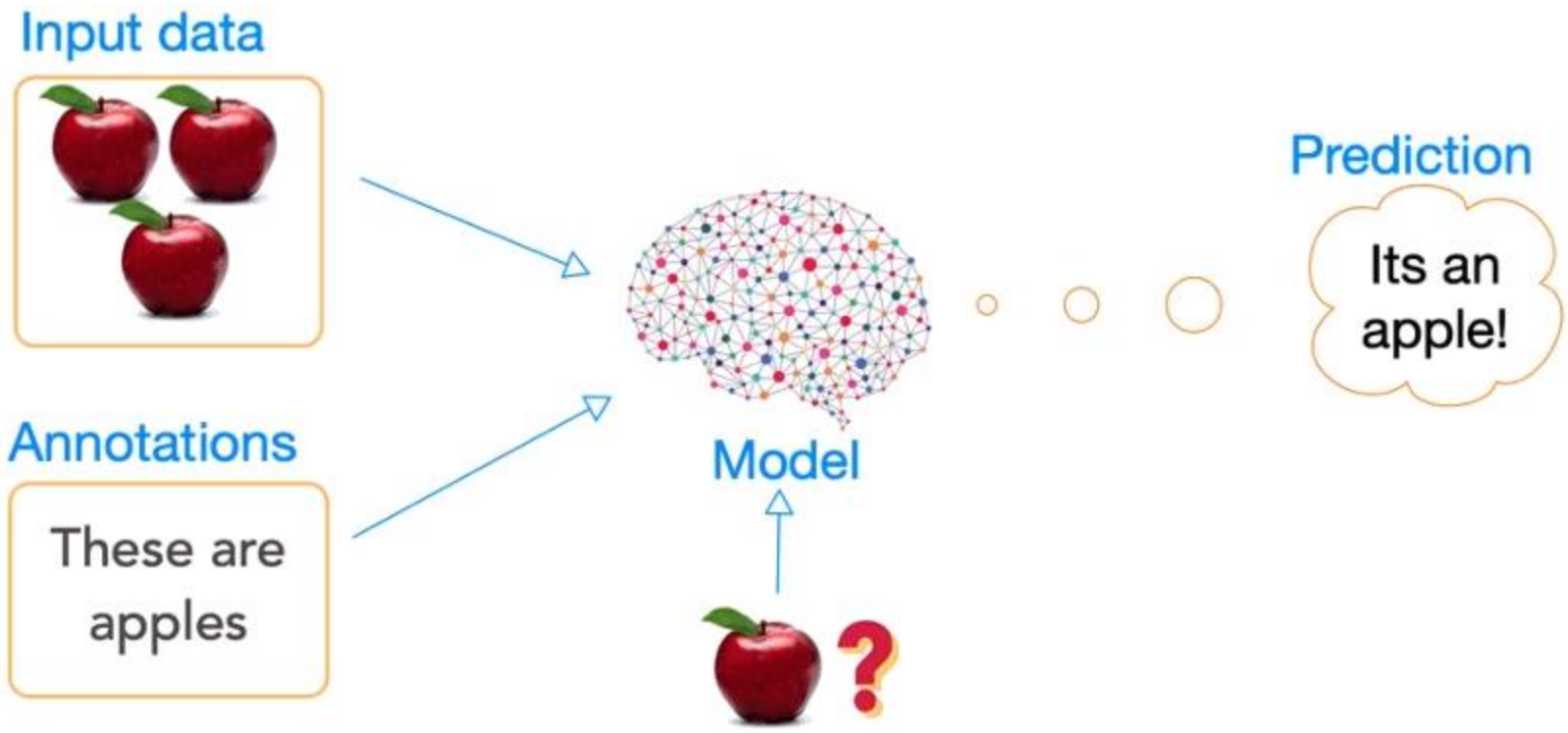
- At the end of this lecture, you should be able to:
  - Understand the function of cluster analysis.
  - Identify the various forms of cluster analysis.
  - Investigate how clustering functions.
  - Implement clustering analysis programming in Python.

Data Models

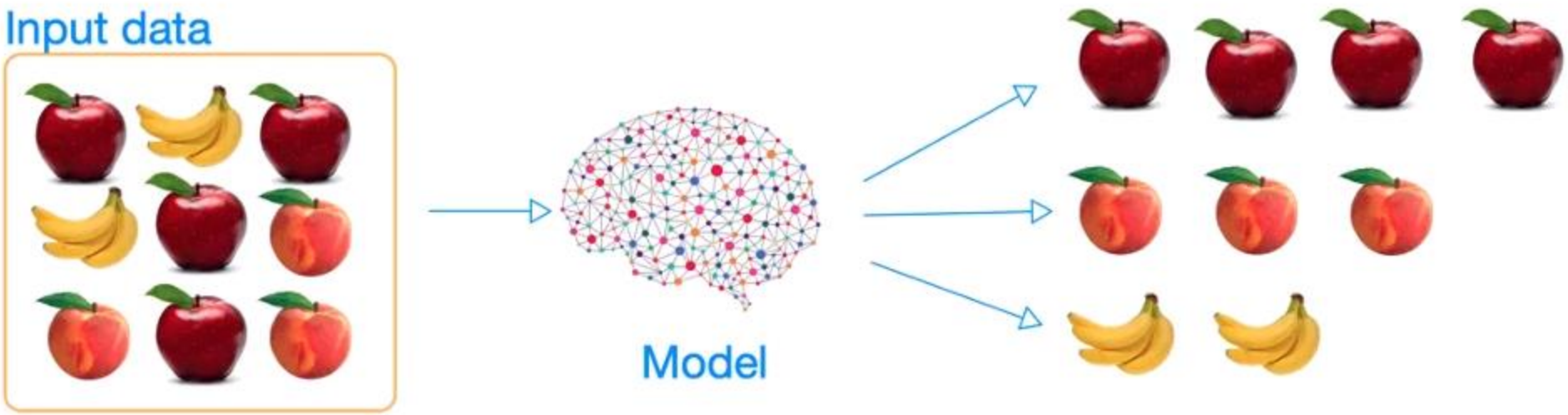
- Data Models

1. Supervised Learning
    1. Regression
    2. Classification
2. Unsupervised Learning
    1. Clustering

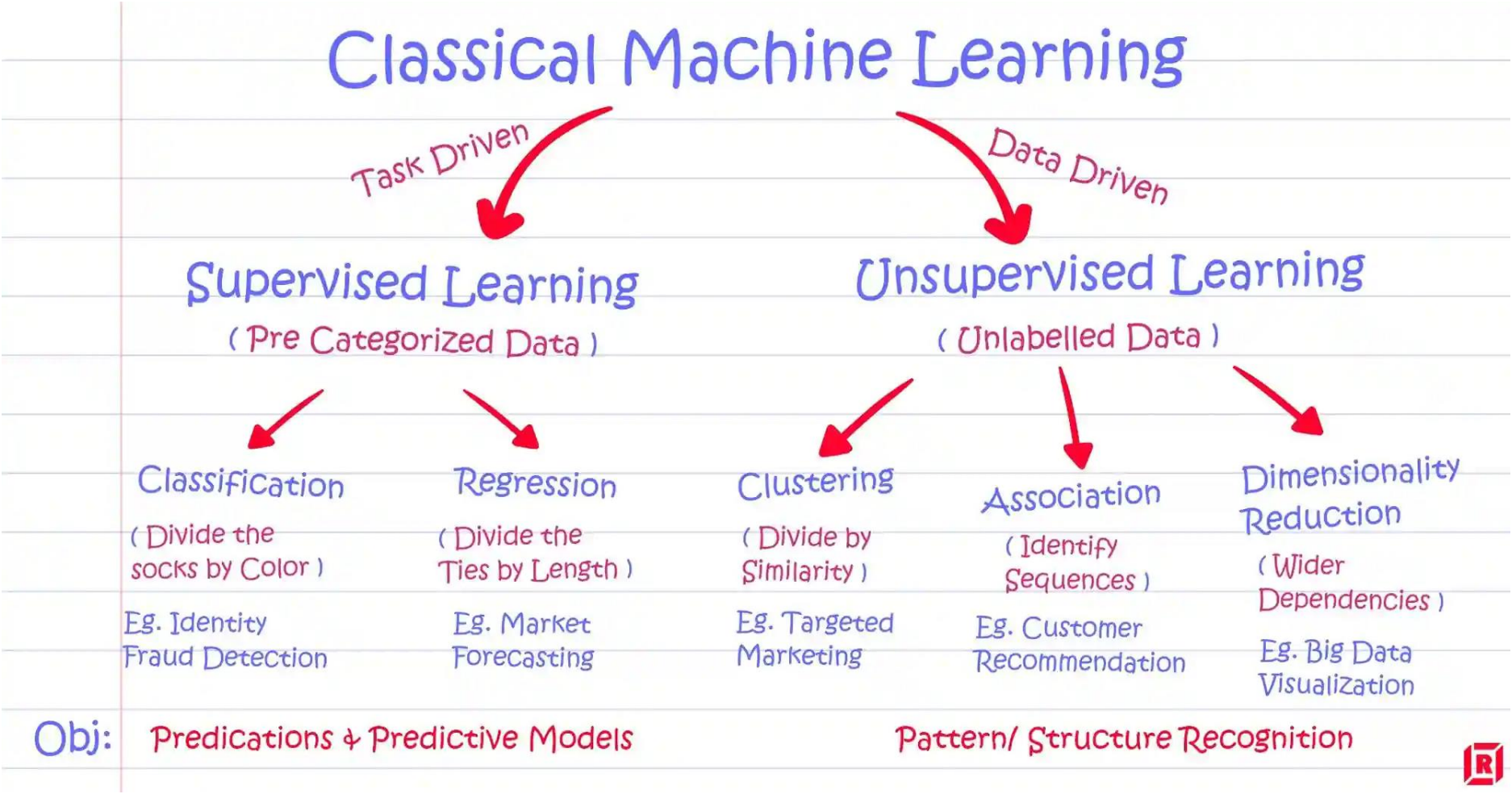
Supervised Learning – Regression, Classification



Unsupervised Learning - Clustering

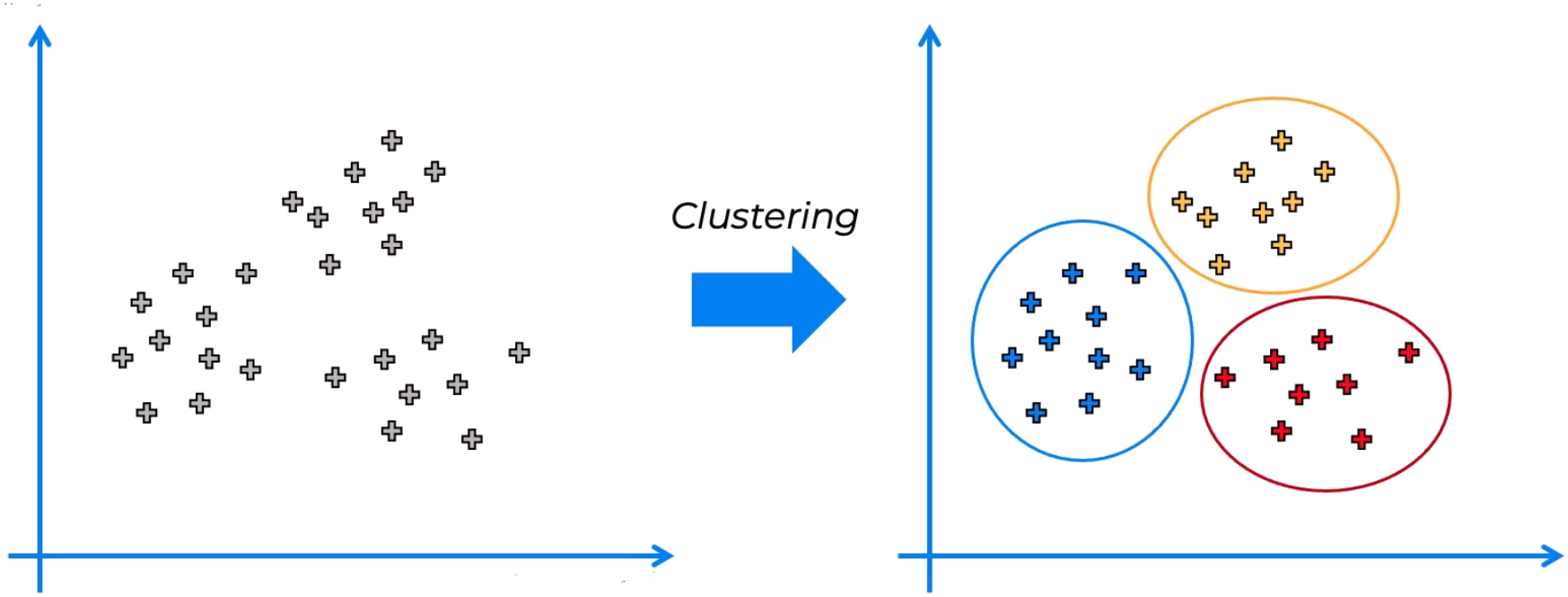


Supervised Vs Unsupervised



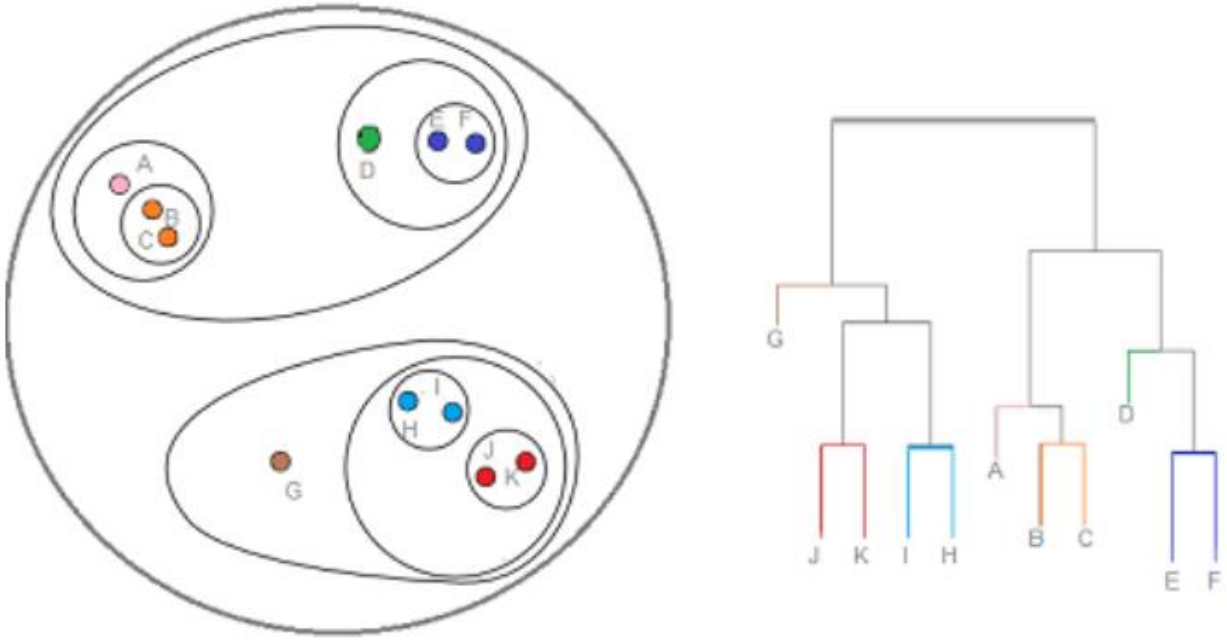
Clustering

- Clustering is a type of unsupervised learning wherein data points are grouped into different sets based on their degree of similarity.



Clustering

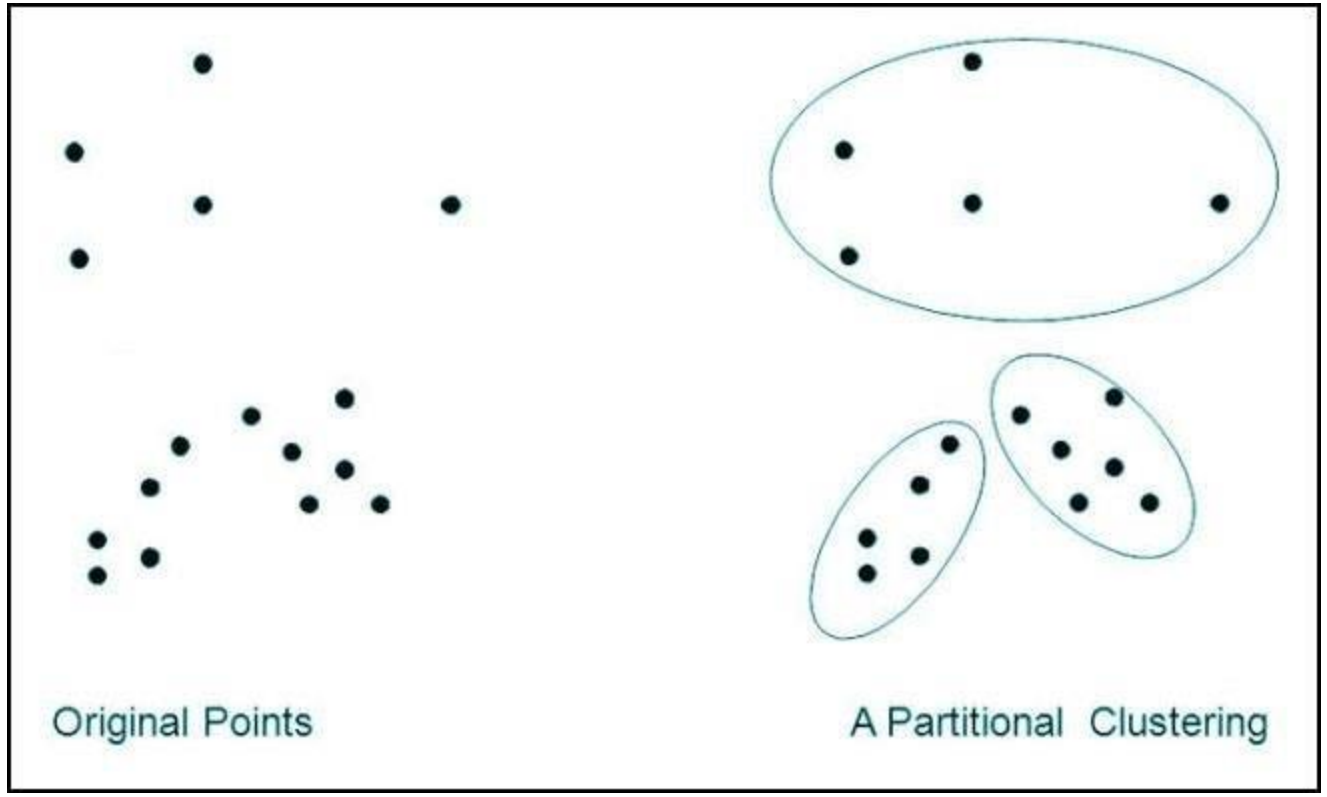
- The various types of clustering are:
  - Hierarchical clustering
  - Partitioning clustering
- **Hierarchical Clustering**
- Builds a hierarchy of clusters, similar to a tree structure.
- Starts with each data point in its own cluster and iteratively merges the most similar clusters until all data points belong to a single cluster.
- It is represented as Dendrograms



A dendrogram (right) representing nested clusters (left).

Clustering

- **Partitioning Clustering**
- Divides the data points into a fixed number of clusters.
- A common algorithm is k-means clustering, which assigns data points to the closest cluster center (centroid).



Clustering

- Hierarchical clustering is further subdivided into:
  - **Agglomerative clustering**
  - Divisive clustering
- Partitioning clustering is further subdivided into:
  - **K-Means clustering**
  - Fuzzy C-Means clustering

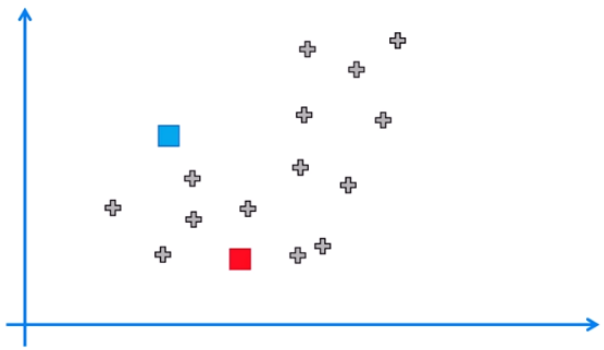
Partitioning Clustering – K-Means

Partitioning Clustering – K-Means

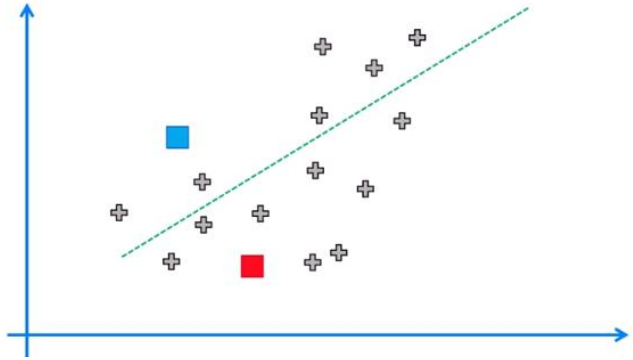
- K-means clustering is used to group similar data points together.
- It is an iterative algorithm that works by
  - assigning **each data point to the closest cluster** centroid.
- The **centroids are then updated** to reflect the new cluster assignments, and
- The **process is repeated** until the centroids converge.

Partitioning Clustering – K-Means

Random Points

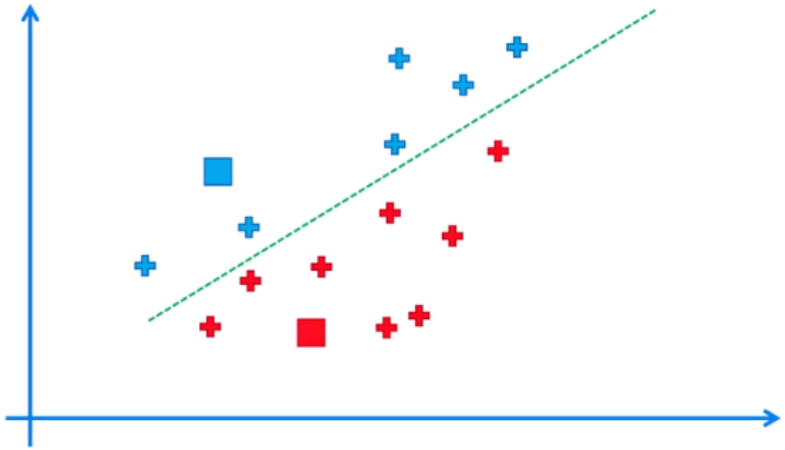


Choose two centroids randomly

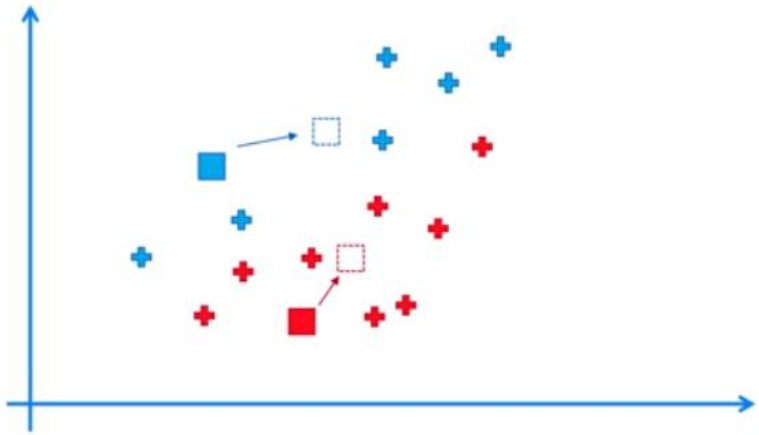


Draw equidistance line between the centroid

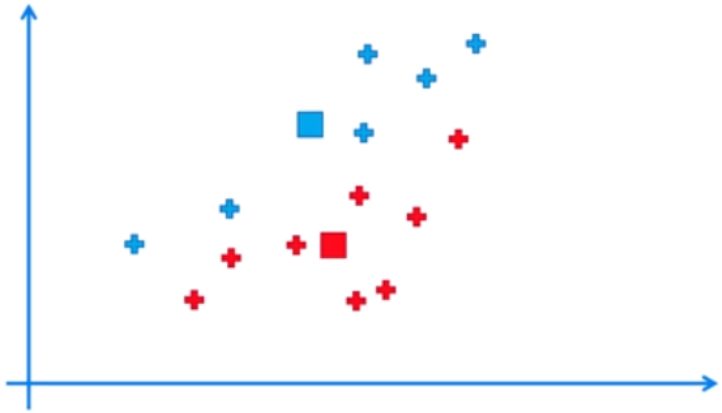
Partitioning Clustering – K-Means



Make a temporary cluster which is above the line and below the line of equidistance

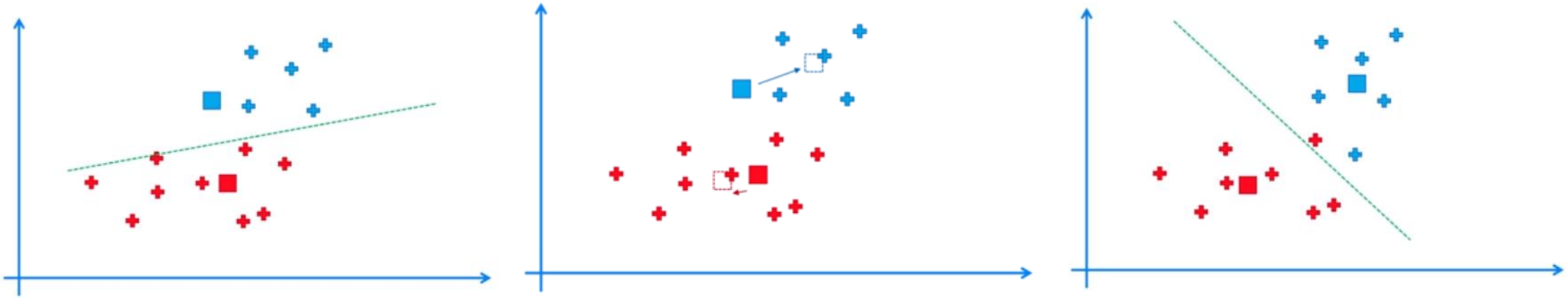


Calculate centroid for each cluster by finding average value of all elements in the cluster

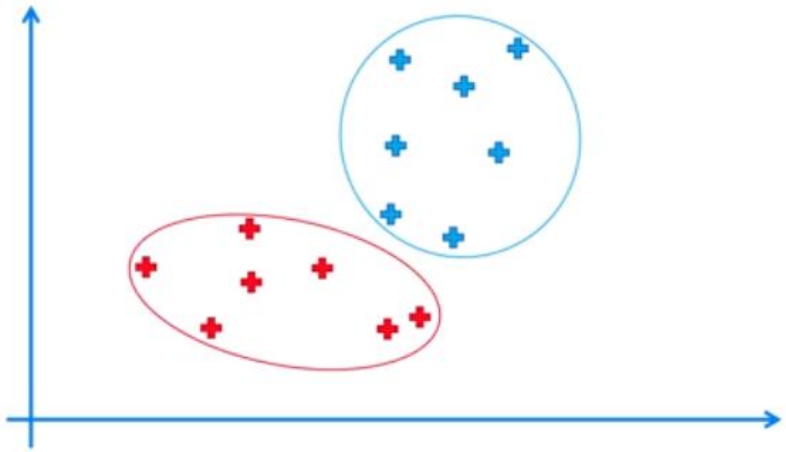


Move the centroid to those position

Partitioning Clustering – K-Means



Repeat the process



Partitioning Clustering – Elbow Method

- The elbow method is a common technique for **determining the optimal number of clusters** (k) in K-means clustering.
- It involves **plotting the within-cluster sum of squares (WCSS) for different values of k** and
- Observing the "elbow" **point where the WCSS starts to decrease at a much slower rate**.

Partitioning Clustering – Elbow Method

- Find the number of clusters in the given dataset using Elbow method

Partitioning Clustering – Elbow Method

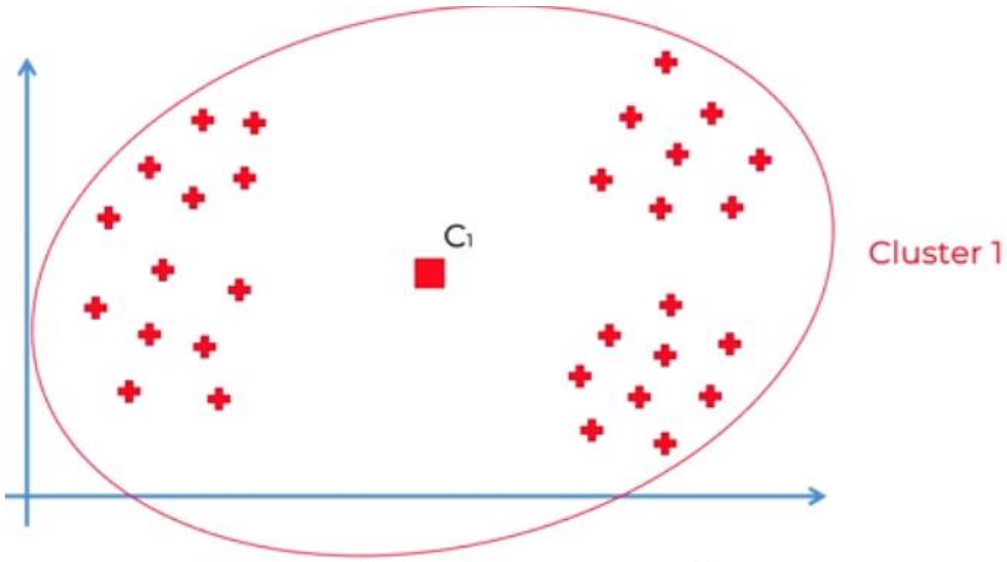
Within Cluster Sum of Squares:

$$\text{WCSS} = \sum_{P_i \text{ in Cluster } 1} \text{distance}(P_i, C_1)^2 + \sum_{P_i \text{ in Cluster } 2} \text{distance}(P_i, C_2)^2 + \ldots$$

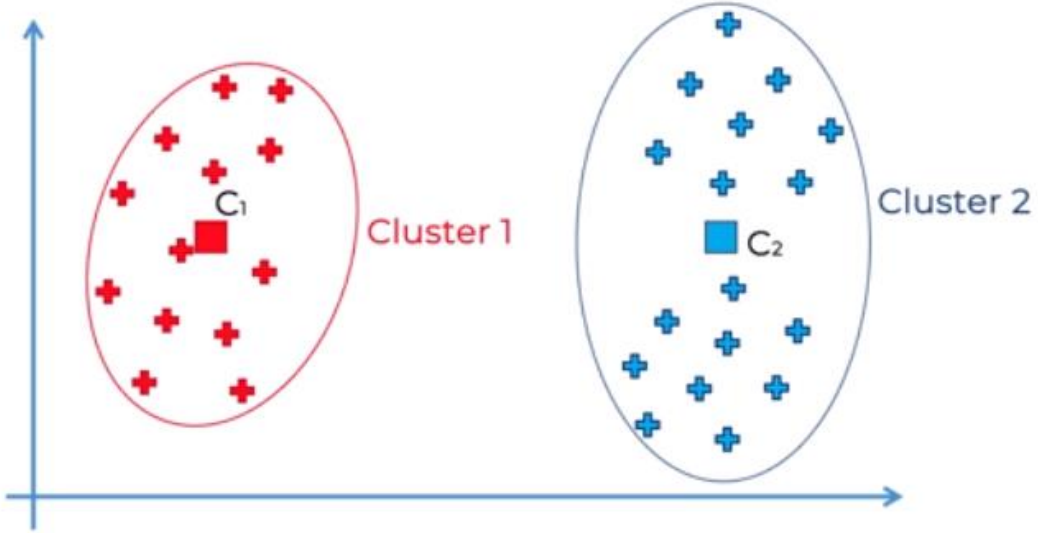
Cluster 1 **Cluster 2**

Partitioning Clustering – Elbow Method

- Apply sum of squares depend on the number of clusters



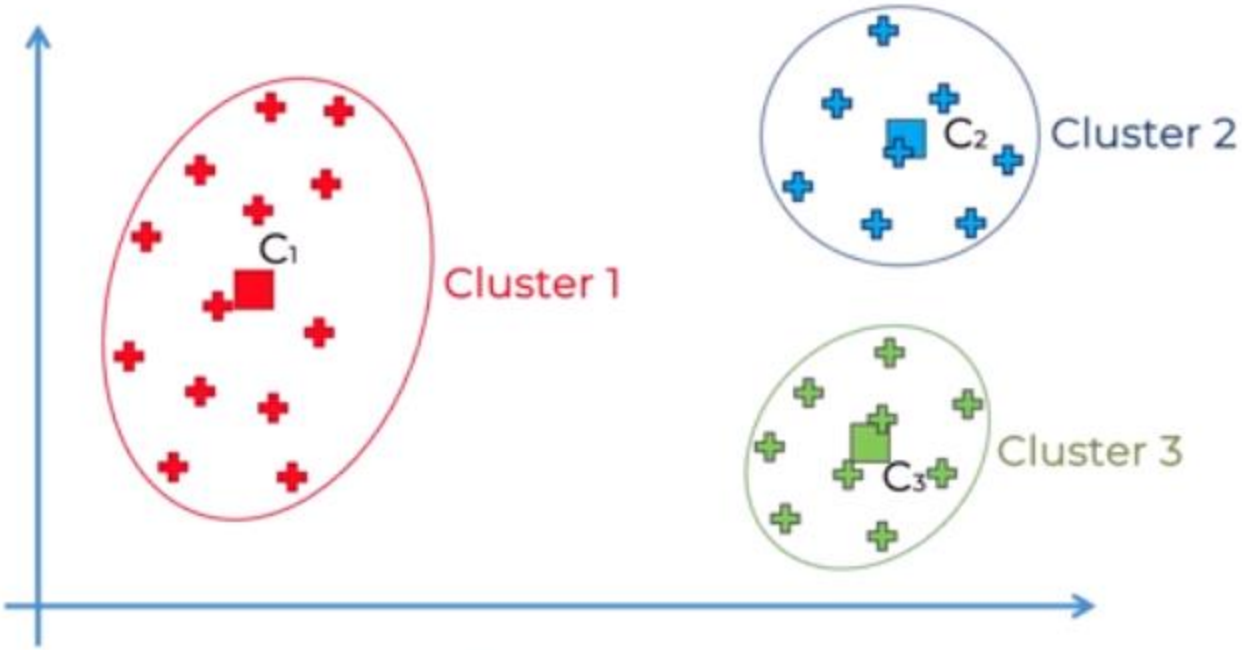
$$\text{WCSS} = \sum_{P_i \text{ in Cluster 1}} \text{distance}(P_i, C_1)^2$$



$$\text{WCSS} = \sum_{P_i \text{ in Cluster 1}} \text{distance}(P_i, C_1)^2 + \sum_{P_i \text{ in Cluster 2}} \text{distance}(P_i, C_2)^2$$

Partitioning Clustering – Elbow Method

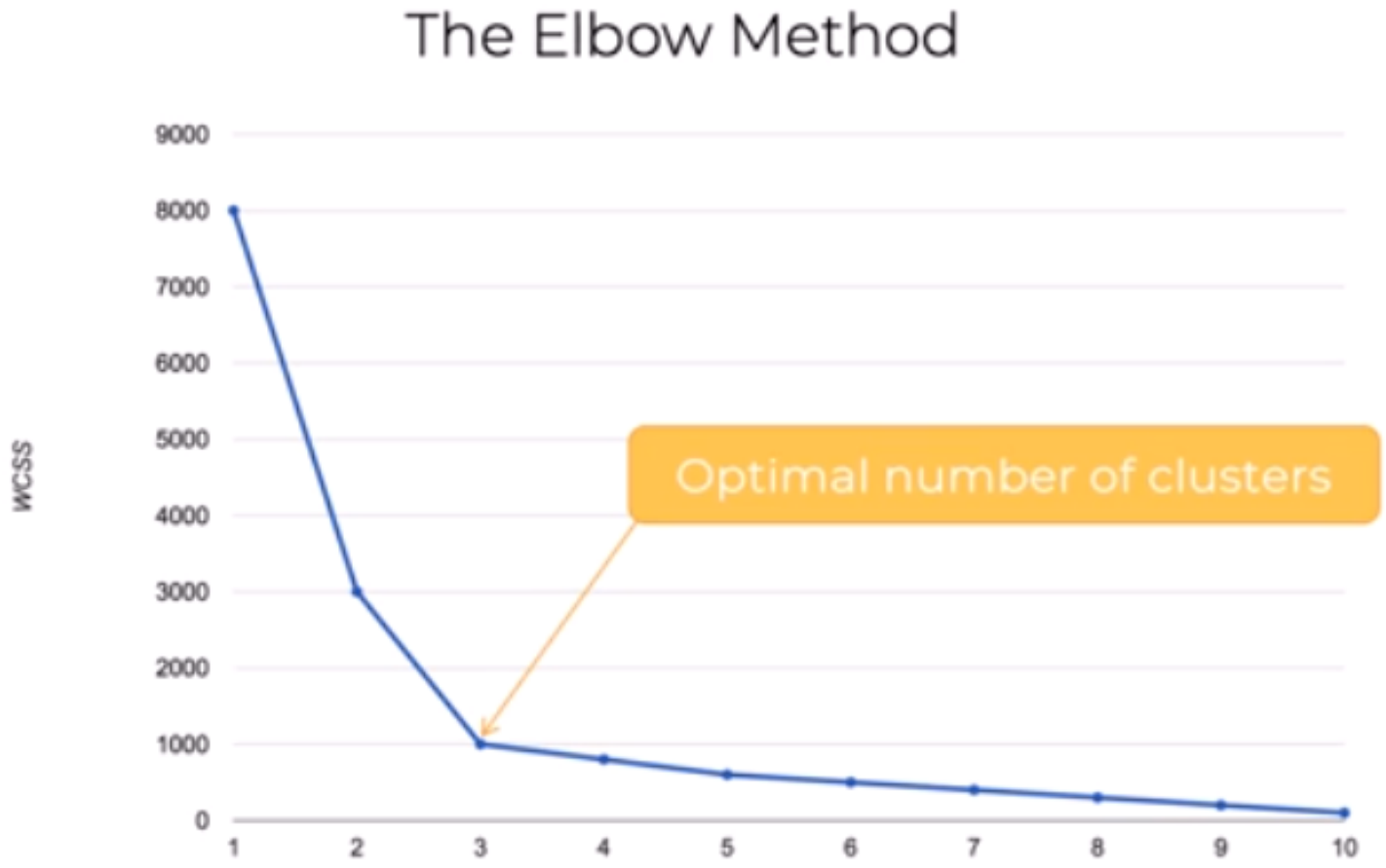
- Apply sum of squares depend on the number of clusters



$$\text{WCSS} = \sum_{P_i \text{ in Cluster 1}} \text{distance}(P_i, C_1)^2 + \sum_{P_i \text{ in Cluster 2}} \text{distance}(P_i, C_2)^2 + \sum_{P_i \text{ in Cluster 3}} \text{distance}(P_i, C_3)^2$$

Partitioning Clustering – Elbow Method

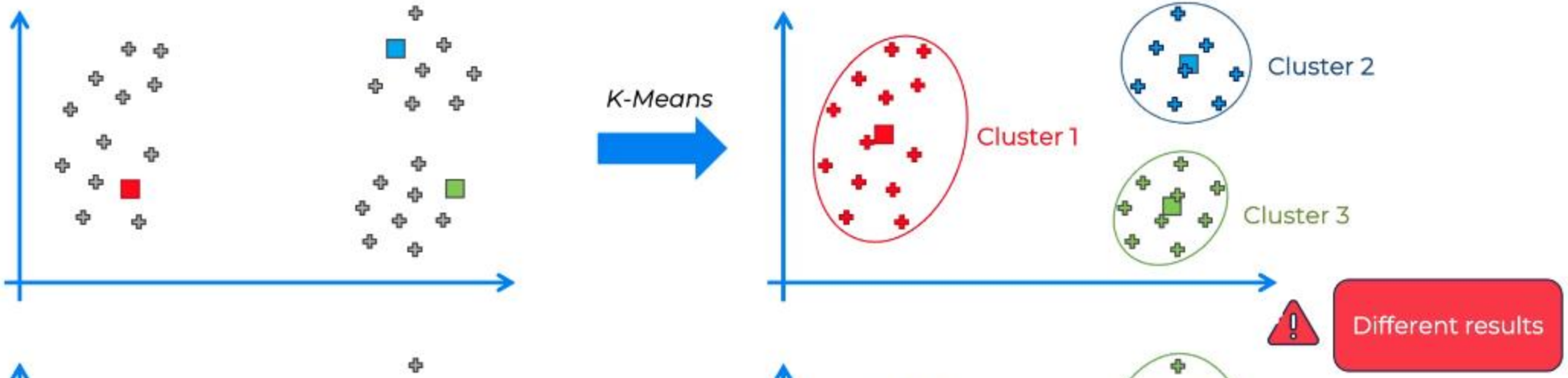
- After calculating WCSS, find the number of clusters you want for the below graph



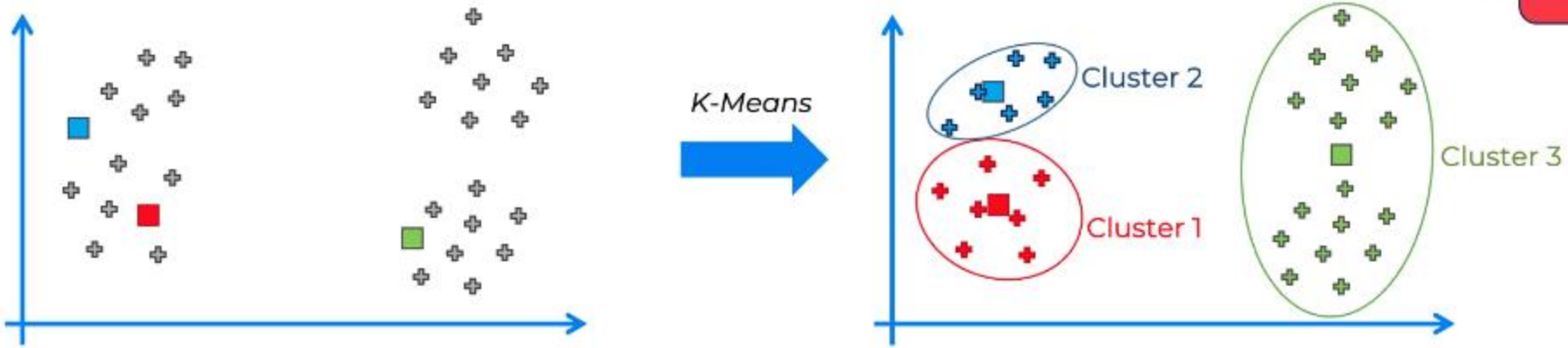
Partitioning Clustering – Issues in K-Means

- Based on placing centroid randomly at the initial stage, the output differs

Case 1



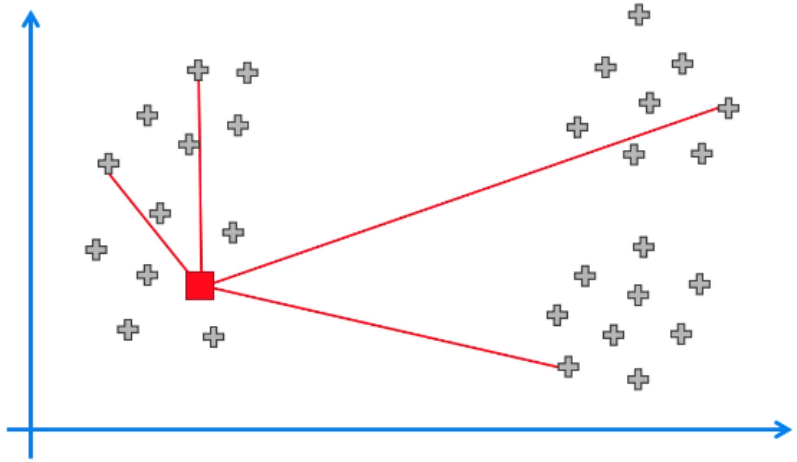
Case 2



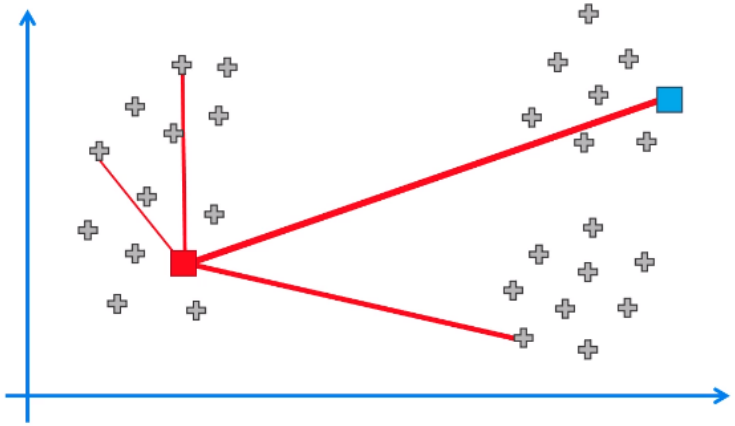
Partitioning Clustering – K-Means++

- Issues of K-Means is solved in K-Means++

Random Points



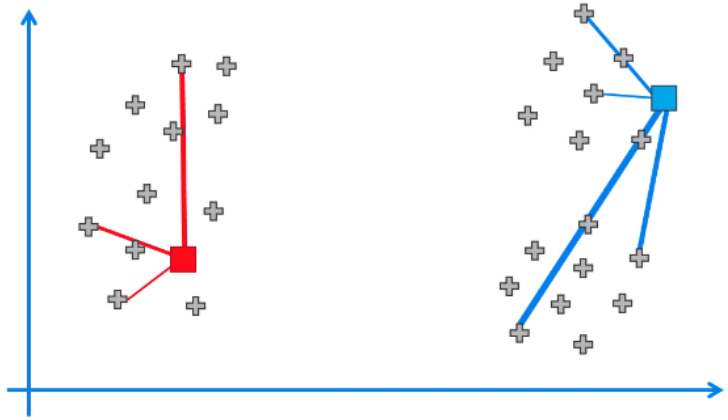
Place centroid randomly and calculate distance between centroid and all the points



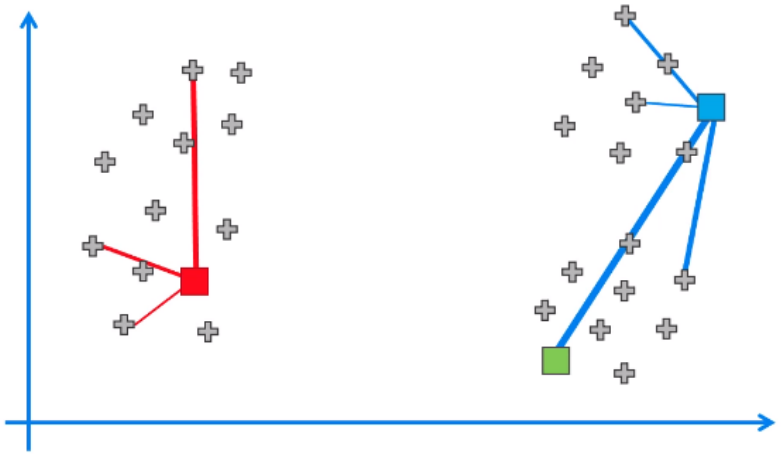
Choose the longest distance and choose as another centroid

Partitioning Clustering – K-Means++

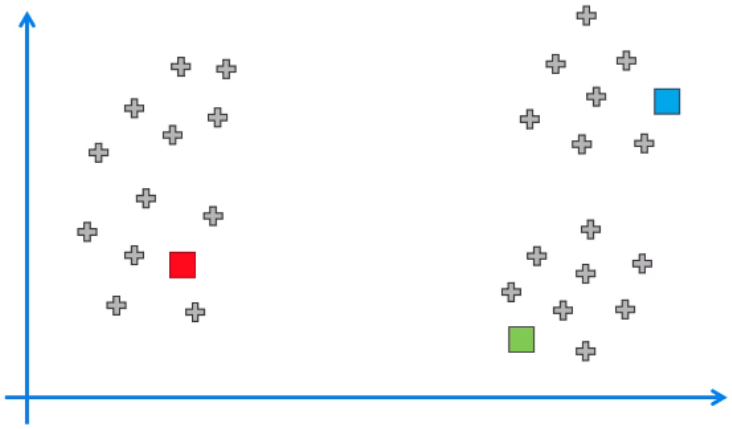
- Issues of K-Means is solved in K-Means++

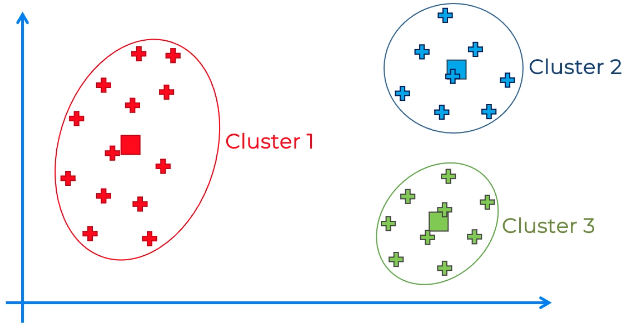


Find the distance between the centroid and other points



Choose the longest distance and choose as another centroid





Build, Train and Find the Optimal Number of Clusters using Partition Clustering – Case Study

Partitioning Clustering - K-Means – Dataset

- Dataset used Mall_Customer.CSV
- From the dataset, we can infer that we can't predict anything but can find some groups or patterns of the customer
- So we can use clustering algorithm instead of regression or classification algorithm

Customer	Genre	Age	Annual Income (k$)	Spending Score (1-100)
1	Male	19	15	39
2	Male	21	15	81
3	Female	20	16	6
4	Female	23	16	77
5	Female	31	17	40
6	Female	22	17	76
7	Female	35	18	6
8	Female	23	18	94
9	Male	64	19	3
10	Female	30	19	72
11	Male	67	19	14
12	Female	35	19	99
13	Female	58	20	15
14	Female	24	20	77
15	Male	37	20	13

K-Means Clustering – Steps in Python

1. Import Libraries:

Python

```python
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
```

2. Prepare Data:

Load the data into a pandas DataFrame or NumPy array and perform any necessary preprocessing.

K-Means Clustering – Steps in Python

3. Calculate WCSS for Different k Values:

Python

```python
wcss_list = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k)
    kmeans.fit(data)
    wcss = kmeans.inertia_
    wcss_list.append(wcss)
```

4. Plot WCSS vs. k:

Python

```python
plt.plot(range(1, 11), wcss_list)
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Within-Cluster Sum of Squares (WCSS)')
plt.title('Elbow Method for Determining Optimal k')
plt.show()
```

K-Means Clustering – Steps in Python

5. Identify the Elbow Point:

Observe the plot and identify the "elbow" point where the WCSS starts to decrease rapidly. The value of k corresponding to this point is considered the optimal number of clusters.

K-Means Clustering – Steps in Python

Steps

1. Importing the libraries
2. Importing the dataset
3. Using the elbow method to find the optimal number of clusters
4. Training the K-Means model on the dataset
5. Visualising the clusters

K-Means Clustering – Steps in Python

Importing the libraries

```
[ ] import numpy as np
    import matplotlib.pyplot as plt
    import pandas as pd
```

Importing the dataset

```
[ ] dataset = pd.read_csv('Mall_Customers.csv')
    X = dataset.iloc[:, [3, 4]].values
```

In cluster analysis, we are not predicting any values. Therefore, separating the dataset's variables into dependent and independent variables is unnecessary.

All rows

We must visualize the cluster at the conclusion of the analysis, which necessitates two columns (X-axis, Y-axis). Therefore, ignore the unimportant field and select the correct column (annual income - 3, expenditure score - 4).

K-Means Clustering – Steps in Python

Using the elbow method to find the optimal number of clusters

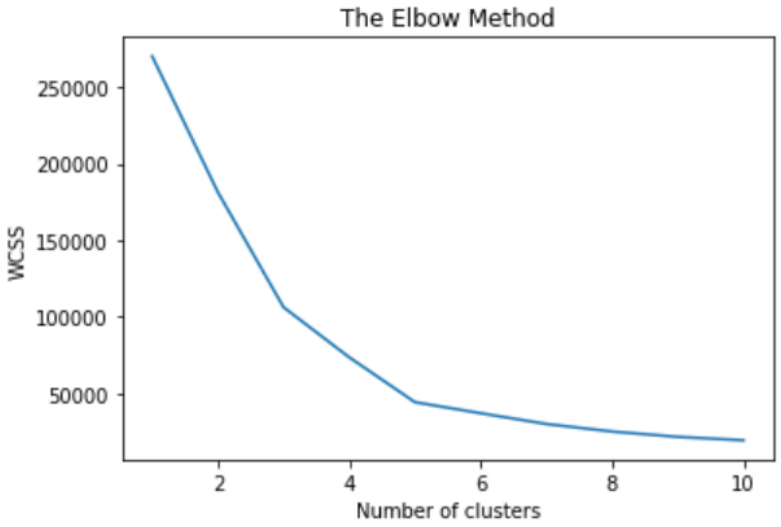
```
from sklearn.cluster import KMeans
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters = i, init = 'k-means++', random_state = 42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)
plt.plot(range(1, 11), wcss)
plt.title('The Elbow Method')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.show()
```

Run the Kmeans algorithm for 10 times

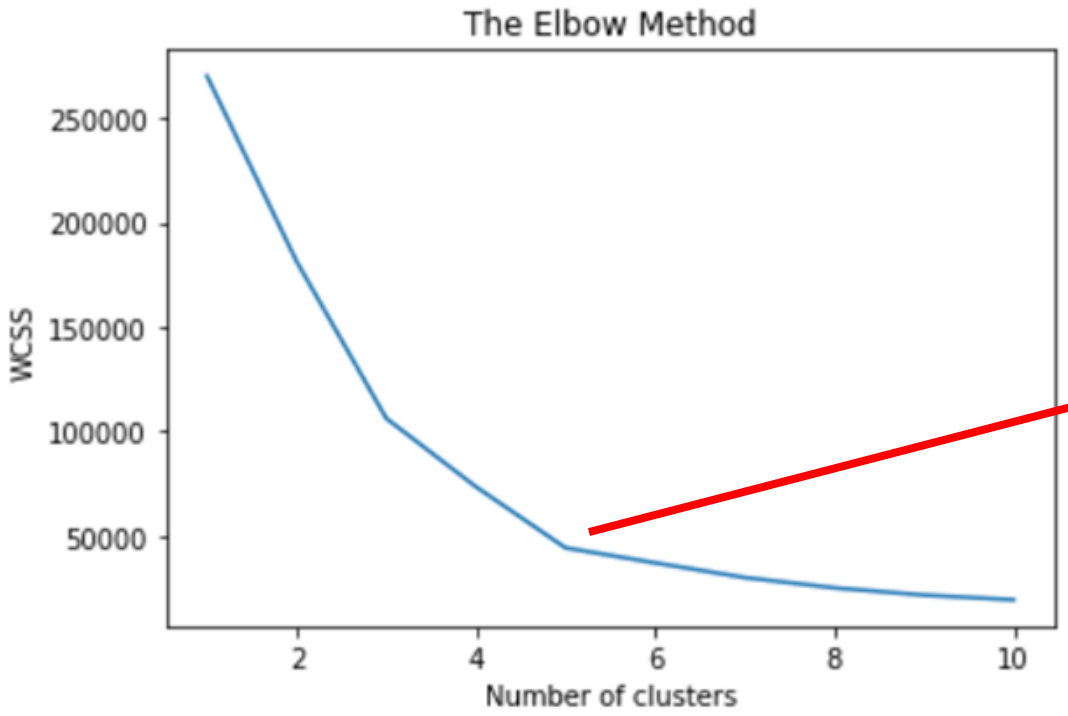
Applying kmeans algorithm to the dataset

After running kmeans, wcss will be obtained which is added to wcss variable

Draw scatter plot for wcss value for each cluster



K-Means Clustering – Steps in Python



Find the number of cluster by analyzing the graph where wcss value reduces at a much slower rate.
Here, when number of cluster=5, wcss reduces at a much slower rate

K-Means Clustering – Steps in Python

Training the K-Means model on the dataset

Number of cluster is obtained from the graph – see previous slide

```
[6] kmeans = KMeans(n_clusters = 5, init = 'k-means++', random_state = 42)
    y_kmeans = kmeans.fit_predict(X)
    print(y_kmeans)
```

Configuration of Kmeans algorithm is attached to the dataset

```
[2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2
 3 2 3 2 3 2 0 2 3 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 4 1 4 0 4 1 4 1 4 0 4 1 4 1 4 1 4 1 4 0 4 1 4 1 4
 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1
 4 1 4 1 4 1 4 1 4 1 4 1 4]
```

Customer id =1 belongs to third cluster

Last Customer belongs to fifth cluster

Customer	Genre	Age	Annual Income (k$)	Spending Score (1-100)
1	Male	19	15	39
2	Male	21	15	81
3	Female	20	16	6
4	Female	23	16	77
5	Female	31	17	40
6	Female	22	17	76
7	Female	35	18	6
8	Female	23	18	94
9	Male	64	19	3
10	Female	30	19	72
11	Male	67	19	14
12	Female	35	19	99
13	Female	58	20	15
14	Female	24	20	77
15	Male	37	20	13

K-Means Clustering – Steps in Python

Visualising the clusters

```
[ ] plt.scatter(X[y_kmeans == 0, 0], X[y_kmeans == 0, 1], s = 100, c = 'red', label = 'Cluster 1')
    plt.scatter(X[y_kmeans == 1, 0], X[y_kmeans == 1, 1], s = 100, c = 'blue', label = 'Cluster 2')
    plt.scatter(X[y_kmeans == 2, 0], X[y_kmeans == 2, 1], s = 100, c = 'green', label = 'Cluster 3')
    plt.scatter(X[y_kmeans == 3, 0], X[y_kmeans == 3, 1], s = 100, c = 'cyan', label = 'Cluster 4')
    plt.scatter(X[y_kmeans == 4, 0], X[y_kmeans == 4, 1], s = 100, c = 'magenta', label = 'Cluster 5')
    plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s = 300, c = 'yellow', label = 'Centroids'
    plt.title('Clusters of customers')
    plt.xlabel('Annual Income (k$)')
    plt.ylabel('Spending Score (1-100)')
    plt.legend()
    plt.show()
```

Scatter plot for first cluster

```
plt.scatter(X[y_kmeans == 0, 0], X[y_kmeans == 0, 1], s = 100, c = 'red', label = 'Cluster 1')
```

X- axis

Y- axis

size

Annual Income (k$)	Spending Score (1-100)
15	39
15	81
16	6
16	77
17	40
17	76
18	6
18	94
19	3
19	72
19	14
19	99
20	15
20	77
20	13

X

First cluster index = 0

```
[2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2 3 2
 3 2 3 2 3 2 0 2 3 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 4 1 4 0 4 1 4 1 4 0 4 1 4 1 4 1 4 1 4 0 4 1 4 1 4
 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4 1
 4 1 4 1 4 1 4 1 4 1 4 1 4 1 4]
```

y_kmeans

K-Means Clustering – Steps in Python

Visualising the clusters

```
[ ] plt.scatter(X[y_kmeans == 0, 0], X[y_kmeans == 0, 1], s = 100, c = 'red', label = 'Cluster 1')
    plt.scatter(X[y_kmeans == 1, 0], X[y_kmeans == 1, 1], s = 100, c = 'blue', label = 'Cluster 2')
    plt.scatter(X[y_kmeans == 2, 0], X[y_kmeans == 2, 1], s = 100, c = 'green', label = 'Cluster 3')
    plt.scatter(X[y_kmeans == 3, 0], X[y_kmeans == 3, 1], s = 100, c = 'cyan', label = 'Cluster 4')
    plt.scatter(X[y_kmeans == 4, 0], X[y_kmeans == 4, 1], s = 100, c = 'magenta', label = 'Cluster 5')
    plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s = 300, c = 'yellow', label = 'Centroids')
    plt.title('Clusters of customers')
    plt.xlabel('Annual Income (k$)')
    plt.ylabel('Spending Score (1-100)')
    plt.legend()
    plt.show()
```

Scatter plot for centroid

```
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s = 300, c = 'yellow', label = 'Centroids'
```

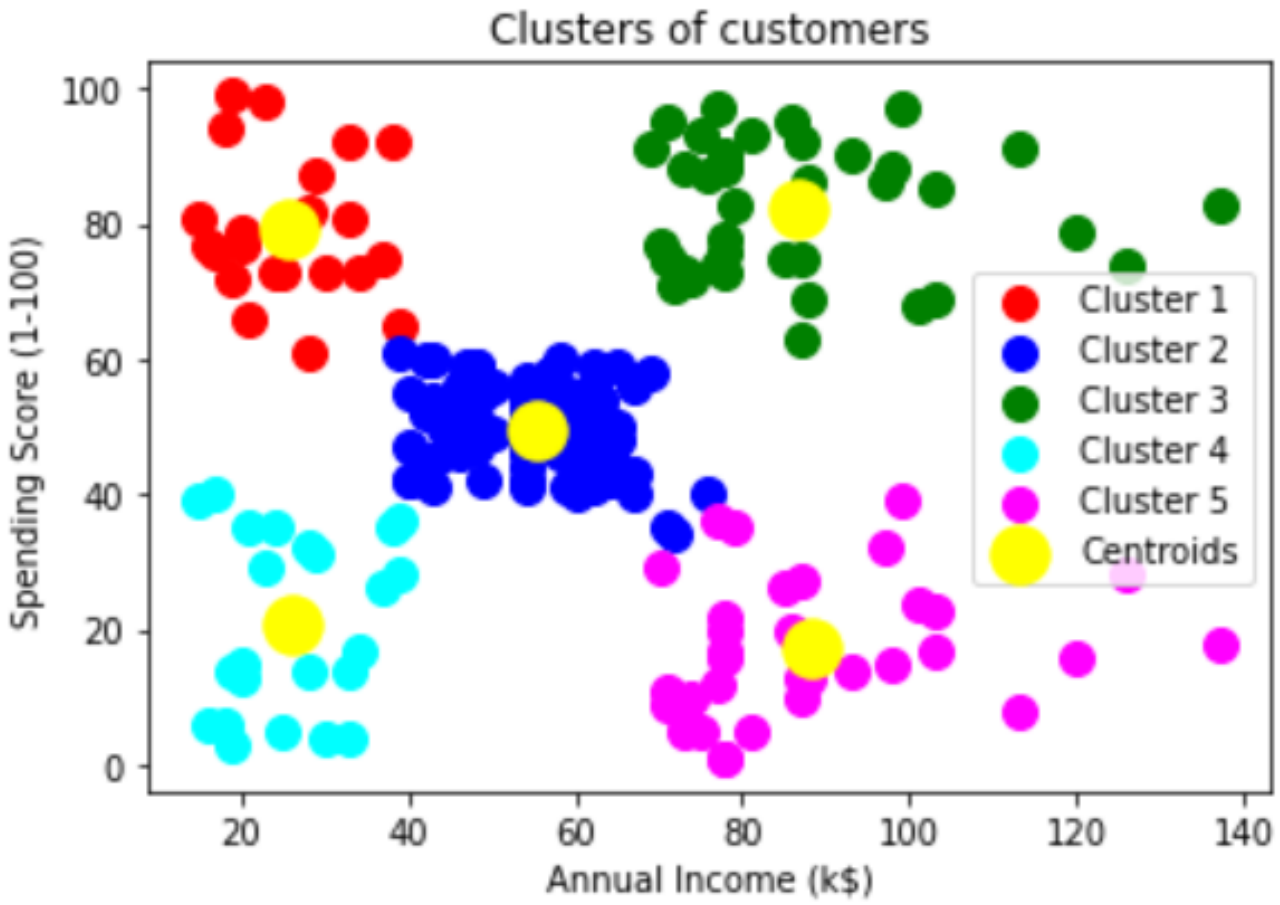
X- axis

Y- axis

size

Annual Income (k$)	Spending Score (1-100)
15	39
15	81
16	6
16	77
17	40
17	76
18	6
18	94
19	3
19	72
19	14
19	99
20	15
20	77
20	13

K-Means Clustering – Steps in Python



K-Means Clustering – Steps in Python

- Sales Strategy

Target this customer since they are having high income and high spending through advertisement

Don't target this customer since they are having low income and high spending

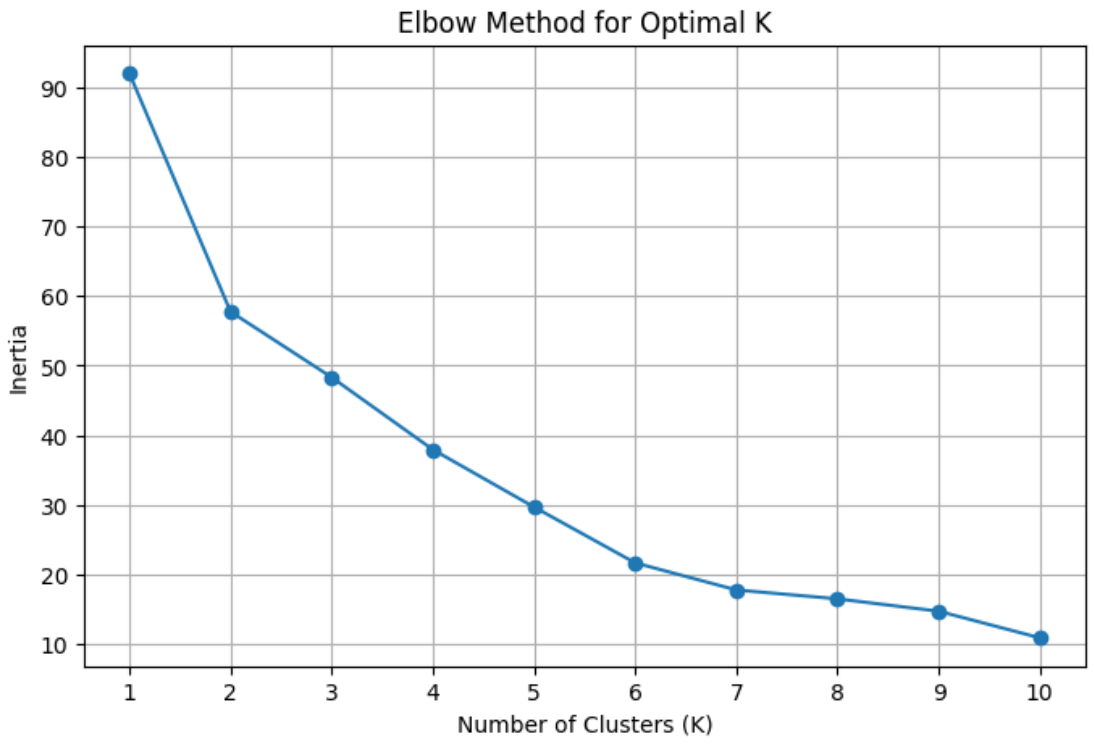
Target this customer since they are having high income and low spending by giving them sales offer

Do It Yourself

- **Perform K-means clustering analysis on the 'Marks' dataset** using 'Quiz', 'Practical', 'Project', and 'Final' marks as features.
- Determine the optimal number of clusters using the elbow method.
- Since visualizing clusters in four dimensions (because of four features) is challenging, apply Principal Component Analysis (PCA) to reduce the dimensionality of the data to two or three dimensions.
- Visualize the clustered data in this reduced space to understand how students are grouped based on their academic performance across the different assessments.

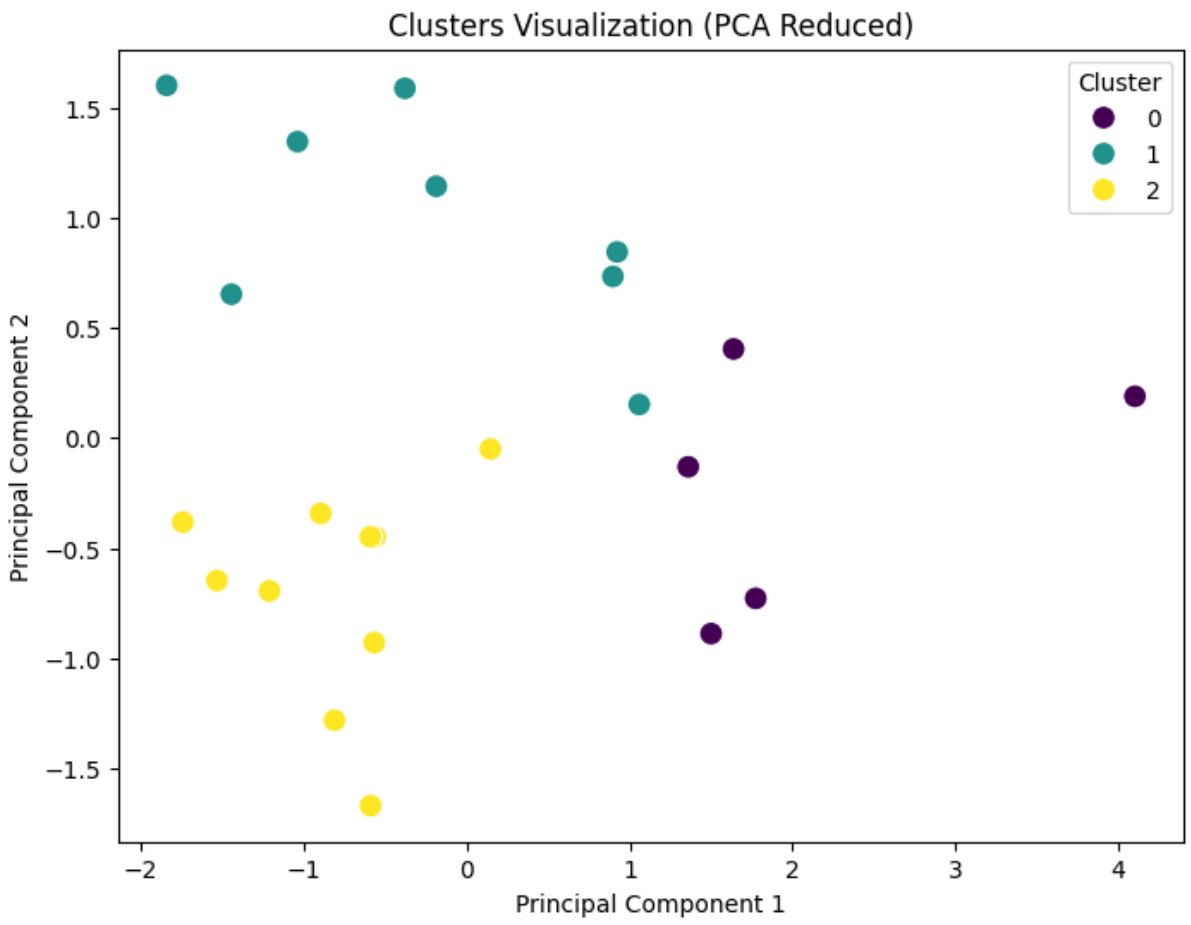
Do It Yourself

- From the attached Elbow Curve, the optimal K-value appears to be 3.
- This is the point where the curve shows a noticeable "elbow," and the rate of decrease in inertia slows down significantly after that.
- Reasoning:
  - For K = 1 to K = 3, there is a steep drop in inertia.
  - After K = 3, the rate of decrease becomes more gradual, indicating diminishing returns from adding more clusters.
- Thus, K = 3 is the most suitable choice for clustering in this case.



Do It Yourself

- **Insights**
  - **Cluster 0**:Could represent students with high scores across all categories.
  - **Cluster 1**:Could represent students with moderate scores in Quiz and Practical but lower scores in Project and Final.
  - **Cluster 2:**Might represent students with low scores across all categories.
- These insights can help identify student performance patterns and inform targeted interventions.



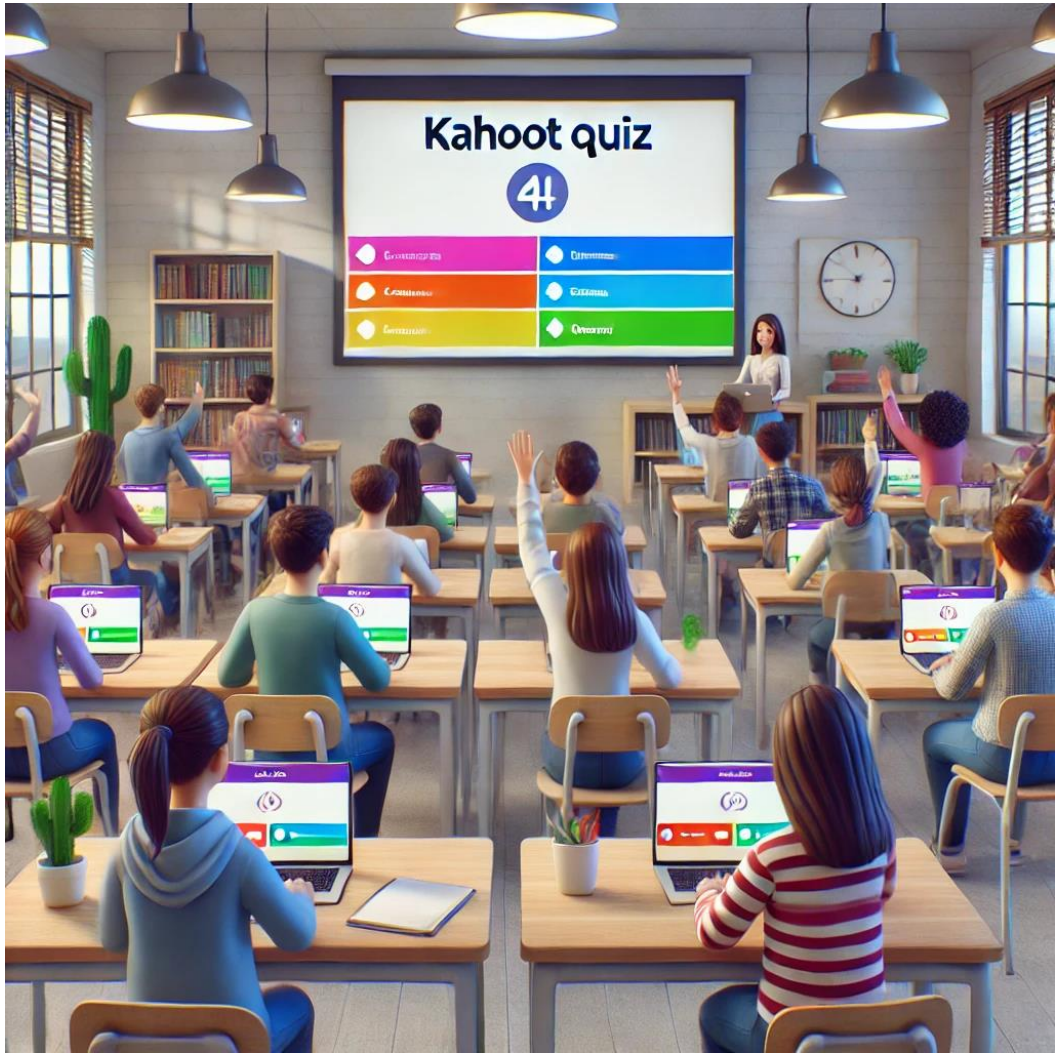
Conclusion

- **Key Takeaways**
- **Clustering** is a fundamental method in **unsupervised learning**.
- It groups data points based on their **similarities**.
- **K-Means Clustering** is widely used due to its simplicity and efficiency.
- **K-Means++** improves initialization, addressing the limitations of standard K-Means.
- **Practical Application**
- **Elbow Method** helps determine the **optimal number of clusters**.
- **Case Study** on the Mall Customer dataset demonstrated real-world application.
- Clustering uncovers patterns and insights in **customer behavior**.

Conclusion

- **Implementation in Python**
- **Step-by-step guide**:
  - **Import libraries**
  - **Load dataset**
  - **Determine optimal clusters**
  - **Train K-Means model**
  - **Visualize clusters**
- Hands-on experience equips students with practical skills.
- **Strategic Insights**
- Clustering informs **business decisions**:
  - **Targeted marketing**
  - **Customer segmentation**
- Enables **data-driven decision-making**.

Kahoot Time!



Higher Colleges of Technology

كليات التقنية العليا

Thank You